

Mohammed Ali Khan

Chicago, IL (Hybrid, Open to Relocate) | connectwithmohammedalikhan@gmail.com | +1 (773) 887-2959 | linkedin.com/in/mohammed-alikhan

PROFESSIONAL SUMMARY

Full Stack Engineer and Scrum Master with 3+ years of experience building event-driven, cloud-native microservices for healthcare claims automation (X12 276/277, 277CA, 835) at Optum (UnitedHealth Group). Design and operate Java 21 / Spring Boot services and Python AI workflow systems on Azure, processing ~300K claims and ~180K Event Hub events daily across 10+ enterprise revenue-cycle customers including Dignity Health. Currently building PAARC, a multi-agent AI service that automates medical insurance appeal processing end to end: LLM-based medical-record structuring, diagnosis reasoning, evidence collection, and appeal-letter drafting using LangGraph, Azure OpenAI, and Azure Document Intelligence behind FHIR R4-compliant APIs. Proven record of cutting API latency 3x, standardizing failure recovery across services, and shipping production AI orchestration (LangGraph) for appeals processing. Lead a 10-engineer Scrum team while maintaining senior-level backend delivery.

SKILLS

Backend & Frameworks: Spring Boot 3.x, Spring Data JPA, Hibernate, Spring WebFlux, Project Reactor, Spring Security (JWT, OAuth2), Spring Retry, Spring Cache (Caffeine), FastAPI, Flask, Uvicorn/Gunicorn, Node.js, Pydantic v2, Drools (rules engine), Apache Velocity

AI & Workflow Orchestration: LangGraph (StateGraph, RetryPolicy, subgraphs), LangChain, Multi-Agent Systems, Azure OpenAI/OpenAI SDK, Azure Document Intelligence (OCR), Agentic Workflows, Durable State Machines, Checkpoint/Resume Based Orchestration, LLM Document Pipelines, Arize OTEL/OpenInference (LLM Observability)

Languages: Java (21), Python, SQL, JavaScript, HTML5, CSS3, C++, C#

Cloud (Azure-Primary): Azure Event Hubs, Azure Service Bus (Session-Enabled Queues, DLQ), Azure Blob Storage, Azure Container Apps, Azure App Service, Azure App Configuration, Azure Key Vault, Azure Managed Identity, AWS (EC2, S3, Lambda), Docker, Kubernetes

Data & Databases: PostgreSQL (JSONB, Partitioning, Batch JDBC), MongoDB / Azure Cosmos DB (Multi-Document Transactions, Compound & TTL indexes, AsyncMongoClient, Aggregation Pipelines, MongoTemplate), SQL Server / Azure SQL, H2, Liquibase, Flyway

APIs & Integration: REST, GraphQL (Spring GraphQL), OpenAPI/Swagger (Springdoc), Webhooks, X12 EDI (276/277, 277CA, 835, 837), HL7 FHIR R4, Event-Driven Architecture, Dead-Letter Queues (DLQ)

Frontend & Mobile: React, React Native (Expo), Tailwind CSS, Bootstrap, jQuery, Thymeleaf

DevOps & Observability: GitHub Actions, SonarQube, JaCoCo, Pytest, Splunk (Dashboards, Alerts, APM), OpenTelemetry, Logback/Logstash Structured Logging, Git, Postman, Insomnia, Docker Desktop, Podman

EXPERIENCE

Full Stack Engineer	Oct 2024 – Present
<i>Optum (UnitedHealth Group)</i>	<i>Schaumburg, IL</i>
- Build and operate the Falcon claim-status platform using Java 21 / Spring Boot 3.x microservices and Python AI services on Azure, automating healthcare claim status, denial, and appeals workflows for 10+ enterprise revenue-cycle customers including Dignity Health, Owensboro, and Northern Lights; also building PAARC, a multi-agent AI service for automated medical insurance appeal processing. - PAARC Appeal Agents (Python, FastAPI, LangGraph, Azure OpenAI) — active development: - Build a multi-agent AI workflow service automating medical insurance appeal processing end to end: OCR and multi-stage LLM structuring of medical-record PDFs (Azure Document Intelligence + Azure OpenAI), diagnosis reasoning, admission-intent extraction, criterion-level clinical evidence collection, and LLM-drafted appeal letters (python-docx) exposed through HL7 FHIR R4-compliant REST APIs (Bundles carrying Task, Questionnaire, and QuestionnaireResponse resources) with Pydantic v2 contracts and MongoDB persistence. - Designed workflow topology as declarative, frozen-dataclass graph specifications (NodeSpec / EdgeSpec / GraphSpec) with startup contract-coverage validation that catches topology and contract drift at deploy time rather than runtime; the 7-node appeal-preprocessing graph orchestrates five protocol-based AI agents covering structuring, reasoning, intent-to-admit, evidence, and letter drafting. - Implemented the evidence engine as a compiled LangGraph StateGraph executing a five-node pipeline per review criterion (cache check, evidence collection, ranking, group evaluation, persistence) with per-node RetryPolicy (3 attempts, exponential backoff) and conditional cache-skip routing, making repeat invocations fully idempotent with zero LLM cost. - Engineered reliable at-least-once processing on session-enabled Azure Service Bus (one session per appeal guaranteeing ordered, single-worker delivery), extending AutoLockRenewer to a 90-minute ceiling for long-running medical-record workloads and building a dedicated DLQ reprocessor that re-enqueues failed messages with resume-mode metadata. - Built workflow resume/skip logic that reconstructs completed activities from audit and checkpoint records on Service Bus redelivery and restores saved node state instead of re-executing, preventing expensive, non-idempotent duplicate LLM calls. - Implemented atomic audit and checkpoint persistence through a decorator-based interceptor that writes both records in a single MongoDB multi-document transaction, including a state sanitizer that converts Python runtime objects (Pydantic models, sets) into	

Mongo-storable primitives.

- Established protocol-based agent interfaces (typing.Protocol) wired through a dependency-injection container, enabling agent implementations to be swapped (stub vs. production LLM) without modifying graph or service code and supporting clean test doubles under pytest.
- Enforced environment-aware security configuration: Managed Identity required and connection strings explicitly forbidden outside local environments, with startup-time ConfigurationError on violation converting credential misconfiguration from a runtime security hole into a deploy-time failure.
- Falcon Claim-Status Platform (Java 21, Spring Boot 3.x, Azure):
- Engineer 6+ event-driven microservices processing ~300K claims across REST and Azure Event Hub ingestion, sustaining ~180K events/day with blob-checkpointed, at-least-once consumers and replay-safe batch processing.
- Reduced latency on timeout-prone API endpoints from ~9s to under 3s by optimizing query patterns, data access paths, and request handling.
- Designed a shared Dead Letter Queue (DLQ) framework adopted by 6 services, standardizing failure capture with SHA-256 payload hashing for deduplication and asynchronous reprocessing APIs, cutting ad-hoc error-handling code by ~40% and eliminating missing-event data loss during downstream outages.
- Built an EDI 277CA acknowledgement pipeline that validates and enriches claim data, generates X12 files with tenant/silo-specific naming, merges processed and unprocessed claim populations behind feature flags, and uploads to Azure Blob Storage via a retry-wrapped, back-pressure-controlled thread pool.
- Developed a follow-up scheduling service encoding multi-branch business rules (response-day sequencing, override dates, payer outage windows, timely-filing deadline clamping) with outbox-style publishing, idempotent batch upserts, and persisted scheduler state to prevent overlapping runs.
- Implemented an 835-remittance denial workflow on Project Reactor with bounded concurrency (15 concurrent events): a 3-stage deterministic claim-matching funnel (winnowing, payment selection, multi-key DB lookup), CARC/RARC denial categorization via cached parallel API calls, and MD5 signature-based deduplication with partition-pruned PostgreSQL queries.
- Delivered exactly-once-like processing on at-least-once infrastructure using PostgreSQL atomic upserts (INSERT ON CONFLICT), canonical event hashing, and transactional DB-write + event-publish boundaries; authored a conditioned JSONB batch-update path (jsonb_set + COALESCE) that prevents null-overwrite corruption of denial dispositions during republish scenarios.
- Built rule-driven workflow orchestration using Drools (DRL) and Velocity templates with strategy-pattern pre-processors and retry evaluators, enabling new payer pathways (IMN, UHC/REAL, OAI) without core code rewrites.
- Hardened Azure integration by wiring user-assigned managed identity, App Configuration, and Key Vault on Azure Container Apps (succeeding after prior DevOps and vendor-supported attempts failed) and implementing Azure AD token injection with connection validation and recycling for resilient Azure SQL access.
- Increased platform resilience with bounded concurrency controls, a dedicated Reactor Netty connection pool for shared WebClient usage (eliminating 502/503/504 pool-exhaustion symptoms), sentinel-driven Caffeine cache refresh via App Configuration, and startup warm-up checks for Event Hub, Blob, and DB dependencies.
- Delivered production observability end to end: comprehensive Splunk logging, business dashboards, OpenTelemetry/Splunk APM distributed tracing with MDC correlation IDs across reactive chains, and alerting that accelerates regression detection and triage.
- Serve as Scrum Master for a 10-engineer team: run sprint planning with directors and product owners, allocate work by capacity, unblock development and testing, and review PRs, while sustaining senior-level individual backend throughput on a biweekly deploy cadence with continuous security and CVE remediation.

Graduate Research Assistant

Feb 2023 – May 2024

Northeastern Illinois University

Chicago, IL

- Led data analysis across 40+ universities and 1,200+ healthcare facilities using Python, Qualtrics, and Excel for a nursing-program feasibility study that directly supported the establishment of NEIU's Nursing Department.
- Automated Illinois Board of Nursing survey workflows in Qualtrics, improving data-collection efficiency by 30%.
- Produced teacher-licensure compliance reporting in TK20 aligned to Illinois State Board of Education requirements, improving submission efficiency by 40%.
- Cleansed and standardized research datasets with Python and Excel, improving data accuracy by ~20%; maintained the college's Drupal website and designed event collateral in Photoshop/Canva.

Technical Support Roles

Jan 2023 – Feb 2023

Northeastern Illinois University

Chicago, IL

- Tutored students in Java, Python, and mathematics coursework and maintained lab and library computer systems; integrated TutorTrac scheduling software for the Learning Success Center, improving appointment-management efficiency by ~20%.

Junior Developer

Oct 2020 – Oct 2021

Next Future IT

Hyderabad, Telangana, India

- Built responsive, scalable web applications with HTML, CSS, JavaScript, and React.js, contributing to a 25% increase in user engagement and 10% lift in conversion rates.

- Implemented Jenkins-based CI and performance optimization, improving page load times by 25% and reducing bounce rate by 15%.
- Practiced Scrum/Kanban delivery, shortening development cycles by 20% and raising sprint velocity by 15%.

PROJECTS

Pennywise (Expense Tracker)

- Java/Spring Boot REST backend with PostgreSQL implementing expense and budget domains; React Native (Expo) mobile UI optimized for fast transaction entry and analytics; GitHub Actions CI with linting and a planned AWS deployment separating backend and client services.

Healthcare Assistant Application

- Real-time health and nutrition assistant built with Python/Flask, React.js, and Firebase, integrating OpenAI GPT-4 Turbo with secure OAuth/Firebase authentication and live chat; sub-2-second response times validated through Jenkins-run unit and integration test suites.

EDUCATION

Northeastern Illinois University

Master of Science in Computer Science. GPA: 4.0.

Chicago, IL

May 2024